

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

Faculty Name: Dr.G.Ayyappan

Student Reg. No: 192571049

Name: Hemalakshmi H

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5

9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Heap File organization (insert, delete, sequential scan)

python

heap_file.py

```
class HeapFile:
```

```
    def __init__(self):
```

```
        self.records = [] # list of dicts: {"id": ..., "name": ...}
```

```
    def insert(self, record):
```

```
        self.records.append(record)
```

```
    def delete(self, key):
```

```
        for i, r in enumerate(self.records):
```

```
            if r["id"] == key:
```

```
                del self.records[i]
```

```
            return True
```

```
        return False
```

```
    def scan(self):
```

```
        for r in self.records:
```

```
            print(r)
```

```
def main():
```

```
    hf = HeapFile()
```

```
    hf.insert({"id": 1, "name": "Alice"})
```

```
    hf.insert({"id": 2, "name": "Bob"})
```

```
    hf.insert({"id": 3, "name": "Charlie"})
```

```
    print("Heap file scan (before delete):")
```

```
    hf.scan()
```

```
    hf.delete(2)
```

```
print("Heap file scan (after delete):")
hf.scan()
```

```
if __name__ == "__main__":
    main()
```

2. Static hashing with chaining (student DB, collisions handled)

python

static_hashing.py

```
class StaticHashTable:
    def __init__(self, size=50):
        self.size = size
        self.table = [[] for _ in range(size)] # chaining

    def hash_func(self, key):
        return key % self.size

    def insert(self, key, value):
        idx = self.hash_func(key)
        self.table[idx].append((key, value))

    def search(self, key):
        idx = self.hash_func(key)
        for k, v in self.table[idx]:
            if k == key:
                return v
        return None

    def display(self):
        for i, chain in enumerate(self.table):
            if chain:
                print(f"Index {i}:", chain)
```

```

def main():
    ht = StaticHashTable(size=50)

    # simulate up to 500 records (here just a few)
    for roll in [101, 151, 201, 251, 301]:
        ht.insert(roll, f'Student_{roll}')

    ht.display()

    print("Search 151:", ht.search(151))
    print("Search 999:", ht.search(999))

if __name__ == "__main__":
    main()

```

3. Sorted file + binary search on primary key

python

sorted_file_binary_search.py

```

class SortedFile:
    def __init__(self):
        self.records = [] # list of dicts with "id"

    def insert(self, record):
        self.records.append(record)
        self.records.sort(key=lambda r: r["id"])

    def binary_search(self, key):
        low, high = 0, len(self.records) - 1
        while low <= high:
            mid = (low + high) // 2

```

```

        mid_id = self.records[mid]["id"]
        if mid_id == key:
            return self.records[mid]
        elif key < mid_id:
            high = mid - 1
        else:
            low = mid + 1
    return None

def display(self):
    for r in self.records:
        print(r)

def main():
    sf = SortedFile()
    sf.insert({"id": 10, "name": "X"})
    sf.insert({"id": 5, "name": "Y"})
    sf.insert({"id": 20, "name": "Z"})

    print("Sorted file contents:")
    sf.display()

    print("Search key 10:", sf.binary_search(10))
    print("Search key 15:", sf.binary_search(15))

if __name__ == "__main__":
    main()

```

4. B-Tree of order 4 (t = 2): insert, search, display

python

btree_order4.py

```

class BTreeNode:
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.children = []

class BTree:
    def __init__(self, t=2): # order 4  $\rightarrow$  t = 2
        self.t = t
        self.root = BTreeNode(leaf=True)

    def search(self, node, key):
        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1
        if i < len(node.keys) and key == node.keys[i]:
            return True
        if node.leaf:
            return False
        return self.search(node.children[i], key)

    def split_child(self, parent, index):
        t = self.t
        node = parent.children[index]
        new_node = BTreeNode(leaf=node.leaf)

        parent.keys.insert(index, node.keys[t - 1])
        parent.children.insert(index + 1, new_node)

        new_node.keys = node.keys[t:]
        node.keys = node.keys[:t - 1]

        if not node.leaf:

```

```
new_node.children = node.children[t:]
node.children = node.children[:t]
```

```
def insert(self, key):
    root = self.root
    if len(root.keys) == 2 * self.t - 1:
        new_root = BTreeNode(leaf=False)
        new_root.children.append(root)
        self.split_child(new_root, 0)
        self.root = new_root
        self._insert_non_full(new_root, key)
    else:
        self._insert_non_full(root, key)
```

```
def _insert_non_full(self, node, key):
    if node.leaf:
        node.keys.append(key)
        node.keys.sort()
    else:
        i = len(node.keys) - 1
        while i >= 0 and key < node.keys[i]:
            i -= 1
        i += 1
        if len(node.children[i].keys) == 2 * self.t - 1:
            self.split_child(node, i)
            if key > node.keys[i]:
                i += 1
        self._insert_non_full(node.children[i], key)
```

```
def display(self, node=None, level=0):
    if node is None:
        node = self.root
    print("Level", level, "Keys:", node.keys)
    if not node.leaf:
```



```

    for child in node.children:
        self.display(child, level + 1)

```

```

def main():
    bt = BTree(t=2)
    keys = [10, 20, 5, 6, 12, 30, 7, 17]
    for k in keys:
        bt.insert(k)

    print("B-Tree structure:")
    bt.display()

    print("Search 12:", bt.search(bt.root, 12))
    print("Search 25:", bt.search(bt.root, 25))

if __name__ == "__main__":
    main()

```

5. B+ Tree with leaf-level linking and range query

python

bplus_tree.py

```

class BPlusNode:
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.next = None # leaf-level link

```

```

class BPlusTree:
    def __init__(self, t=2):

```

```

self.t = t
self.root = BPlusNode(leaf=True)

def insert(self, key):
    root = self.root
    if len(root.keys) == 2 * self.t - 1:
        new_root = BPlusNode(leaf=False)
        new_root.children.append(root)
        self.split_child(new_root, 0)
        self.root = new_root
        self._insert_non_full(new_root, key)
    else:
        self._insert_non_full(root, key)

def split_child(self, parent, index):
    t = self.t
    node = parent.children[index]
    new_node = BPlusNode(leaf=node.leaf)

    mid = t - 1
    parent.keys.insert(index, node.keys[mid])
    parent.children.insert(index + 1, new_node)

    new_node.keys = node.keys[mid:]
    node.keys = node.keys[:mid]

    if node.leaf:
        new_node.next = node.next
        node.next = new_node

def _insert_non_full(self, node, key):
    if node.leaf:
        node.keys.append(key)
        node.keys.sort()

```

```

else:
    i = len(node.keys) - 1
    while i >= 0 and key < node.keys[i]:
        i -= 1
    i += 1
    self._insert_non_full(node.children[i], key)

```

```

def range_query(self, start, end):

```

```

    node = self.root
    while not node.leaf:
        node = node.children[0]

```

```

    result = []
    while node:
        for k in node.keys:
            if start <= k <= end:
                result.append(k)
        node = node.next
    return result

```

```

def display_leaves(self):

```

```

    node = self.root
    while not node.leaf:
        node = node.children[0]
    print("Leaf level:")
    while node:
        print(node.keys, "->", end=" ")
        node = node.next
    print("None")

```

```

def main():

```

```

    bpt = BPlusTree(t=2)
    keys = [10, 20, 5, 6, 12, 30, 7, 17]

```

```
for k in keys:
    bpt.insert(k)
```

```
bpt.display_leaves()
print("Range [6, 17]:", bpt.range_query(6, 17))
```

```
if __name__ == "__main__":
    main()
```

6. Dense primary index on a sorted file

python

dense_primary_index.py

```
class SortedFile:
```

```
    def __init__(self):
        self.records = []
```

```
    def insert(self, record):
        self.records.append(record)
        self.records.sort(key=lambda r: r["id"])
```

```
class DenseIndex:
```

```
    def __init__(self, sorted_file):
        self.index = [(rec["id"], pos) for pos, rec in enumerate(sorted_file.records)]
```

```
    def search(self, key):
        for k, pos in self.index:
            if k == key:
                return pos
        return None
```

```
    def display(self):
```

```

        print("Dense index table (key -> position):")
        for k, pos in self.index:
            print(k, "->", pos)

def main():
    sf = SortedFile()
    sf.insert({"id": 5, "name": "A"})
    sf.insert({"id": 10, "name": "B"})
    sf.insert({"id": 20, "name": "C"})

    di = DenseIndex(sf)
    di.display()

    key = 10
    pos = di.search(key)
    if pos is not None:
        print(f"Record for key {key}:", sf.records[pos])
    else:
        print("Key not found")

if __name__ == "__main__":
    main()

```

7. Extendible hashing with directory doubling

python

extendible_hashing.py

```

class Bucket:
    def __init__(self, depth):
        self.depth = depth
        self.values = []

```

```

class ExtendibleHash:
    def __init__(self, bucket_capacity=2):
        self.global_depth = 1
        self.bucket_capacity = bucket_capacity
        self.directory = [Bucket(depth=1), Bucket(depth=1)]

    def hash_bits(self, key):
        return key & ((1 << self.global_depth) - 1)

    def insert(self, key):
        idx = self.hash_bits(key)
        bucket = self.directory[idx]

        if len(bucket.values) < self.bucket_capacity:
            bucket.values.append(key)
        else:
            self.split_bucket(idx)
            self.insert(key)

    def split_bucket(self, idx):
        bucket = self.directory[idx]
        bucket.depth += 1

        if bucket.depth > self.global_depth:
            self.global_depth += 1
            self.directory += [Bucket(depth=b.depth) for b in self.directory]

        old_values = bucket.values
        bucket.values = []
        for v in old_values:
            new_idx = self.hash_bits(v)
            self.directory[new_idx].values.append(v)

```

```

def display(self):
    print("Global depth:", self.global_depth)
    for i, b in enumerate(self.directory):
        print(f"Dir {i}: depth={b.depth}, values={b.values}")

def main():
    eh = ExtendibleHash(bucket_capacity=2)
    keys = [5, 7, 9, 12, 15, 18, 20, 25] # 8 records
    for k in keys:
        eh.insert(k)

    eh.display()

if __name__ == "__main__":
    main()

```

8. Secondary storage block access: sequential vs indexed I/O

python

secondary_storage_io.py

```

def compute_blocks(num_records, block_size):
    return (num_records + block_size - 1) // block_size

def main():
    num_records = 1000
    block_size = 100 # records per block

    sequential_blocks = compute_blocks(num_records, block_size)
    sequential_io = sequential_blocks # one I/O per block

    indexed_io = 2 # 1 I/O for index block + 1 for data block (simplified model)

```

```

print("Number of records:", num_records)
print("Block size:", block_size)
print("Sequential retrieval I/Os:", sequential_io)
print("Indexed retrieval I/Os:", indexed_io)

```

```

if __name__ == "__main__":
    main()

```

9: Sparse Index + Dense Index + Timing Analysis

python

sparse_vs_dense_index.py

import time

import random

Sorted File

class SortedFile:

def __init__(self):

self.records = []

def insert(self, record):

self.records.append(record)

self.records.sort(key=lambda r: r["id"])

Dense Index

class DenseIndex:

def __init__(self, sorted_file):

self.index = [(rec["id"], pos) for pos, rec in enumerate(sorted_file.records)]


```

def search(self, key):
    for k, pos in self.index:
        if k == key:
            return pos
    return None

```

```

class SparseIndex:

```

```

    def __init__(self, sorted_file, block_size=50):
        self.block_size = block_size
        self.index = [] # (key, block_start_position)

        for i in range(0, len(sorted_file.records), block_size):
            key = sorted_file.records[i]["id"]
            self.index.append((key, i))

```

```

    def search(self, sorted_file, key):
        # find block
        block_pos = None
        for k, pos in self.index:
            if key >= k:
                block_pos = pos

        if block_pos is None:
            return None

        # linear scan inside block
        end = min(block_pos + self.block_size, len(sorted_file.records))
        for i in range(block_pos, end):
            if sorted_file.records[i]["id"] == key:
                return i
        return None

```

```

def main():
    sf = SortedFile()

    # create 500 sorted records
    for i in range(1, 501):
        sf.insert({"id": i, "name": f"Student_{i}"})

    dense = DenseIndex(sf)
    sparse = SparseIndex(sf, block_size=50)

    search_key = random.randint(1, 500)

    # Dense index timing
    t1 = time.time()
    pos_dense = dense.search(search_key)
    t2 = time.time()

    # Sparse index timing
    t3 = time.time()
    pos_sparse = sparse.search(sf, search_key)
    t4 = time.time()

    print("Search Key:", search_key)
    print("Dense Index Position:", pos_dense, "Time:", (t2 - t1))
    print("Sparse Index Position:", pos_sparse, "Time:", (t4 - t3))

if __name__ == "__main__":
    main()

```

Program 10: B+ Tree + Range Query

python

bplus_tree_range_query.py

```

class BPlusNode:
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.next = None # leaf link


class BPlusTree:
    def __init__(self, t=2):
        self.t = t
        self.root = BPlusNode(leaf=True)

    def insert(self, key):
        root = self.root
        if len(root.keys) == 2 * self.t - 1:
            new_root = BPlusNode(leaf=False)
            new_root.children.append(root)
            self.split_child(new_root, 0)
            self.root = new_root
            self._insert_non_full(new_root, key)
        else:
            self._insert_non_full(root, key)

    def split_child(self, parent, index):
        t = self.t
        node = parent.children[index]
        new_node = BPlusNode(leaf=node.leaf)

        mid = t - 1
        parent.keys.insert(index, node.keys[mid])
        parent.children.insert(index + 1, new_node)

        new_node.keys = node.keys[mid:]

```

```
node.keys = node.keys[:mid]
```

```
if node.leaf:
```

```
    new_node.next = node.next
```

```
    node.next = new_node
```

```
def _insert_non_full(self, node, key):
```

```
    if node.leaf:
```

```
        node.keys.append(key)
```

```
        node.keys.sort()
```

```
    else:
```

```
        i = len(node.keys) - 1
```

```
        while i >= 0 and key < node.keys[i]:
```

```
            i -= 1
```

```
        i += 1
```

```
        self._insert_non_full(node.children[i], key)
```

```
def traverse(self):
```

```
    print("B+ Tree (Level Order):")
```

```
    q = [self.root]
```

```
    level = 0
```

```
    while q:
```

```
        next_q = []
```

```
        print("Level", level, ":", end=" ")
```

```
        for node in q:
```

```
            print(node.keys, end=" | ")
```

```
            if not node.leaf:
```

```
                next_q.extend(node.children)
```

```
        print()
```

```
        q = next_q
```

```
        level += 1
```

```
def range_query(self, start, end):
```

```
    node = self.root
```

```
while not node.leaf:
    node = node.children[0]
```

```
result = []
while node:
    for k in node.keys:
        if start <= k <= end:
            result.append(k)
    node = node.next
return result
```

```
def main():
    bpt = BPlusTree(t=2)

    keys = [5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
    for k in keys:
        bpt.insert(k)

    bpt.traverse()

    print("Range Query 15 to 45:")
    print(bpt.range_query(15, 45))

if __name__ == "__main__":
    main()
```